



Robotics and Navigation in Medicine

Course Project: "Image Guided Robotic Needle Placement"

Group Members

Ashutosh Goyal (Group Leader)

Ravi Prakash Dhorajia (Planning Team Leader)

Shashank Somasundaram

Sharjeel Ahmad

Vinit Harshesh Trivedi



Under The Guidance Of:

Prof. Alexander Schlaefer

Tutors: Konrad Reuter & Michael Meyling

1. Abstract

Minimally invasive procedures such as biopsies and targeted drug delivery rely on precise and safe medical needle placement. Manual insertions are prone to variability due to human factors, whereas robotic systems provide the potential for enhanced accuracy, repeatability, and safety. In this project, we developed an image-guided robotic needle placement system using the Robot Operating System (ROS) framework. A Franka Emika Panda robotic arm was equipped with a Microsoft Azure Kinect depth camera to acquire 3D scans of a chest phantom. Through eye-in-hand calibration, we estimated the transformation between the robot end-effector and the mounted camera, enabling multi-view image stitching and point cloud registration. These scans were aligned with a high-resolution computed tomography (CT)-derived phantom model, within which target locations for needle insertion were defined. Inverse kinematics and trajectory planning were implemented to compute collision-free and kinematically feasible insertion paths. Finally, the camera was replaced with a mock-up needle, and the robot executed the insertion task. This paper describes the methodology, system integration, and experimental results of the project, with emphasis on calibration accuracy, trajectory planning, and the robustness of the insertion procedure.

2. Introduction

Medical needle placement is a critical component of many minimally invasive clinical procedures, including biopsies, ablations, and targeted drug delivery. While manual placement by clinicians is widely practiced, its accuracy is often limited by anatomical complexity, operator experience, and imaging constraints. Robotic assistance in needle placement has emerged as a promising approach to overcome these limitations, offering improved precision, consistency, and the potential for reduced procedural risks.

In this project, we aimed to design, implement, and evaluate an image-guided robotic needle placement system using ROS. The system integrated computer vision, robotic kinematics, trajectory planning, and calibration methods to achieve accurate targeting within a chest phantom model. The Franka Emika Panda robot (see Figure 1) served as the manipulator, while a Microsoft Azure Kinect provided 3D imaging for model registration. The project required the development of modules for robot kinematics, trajectory planning, robotic scanning, needle calibration, camera calibration, eye-in-hand calibration, and 3D model registration.

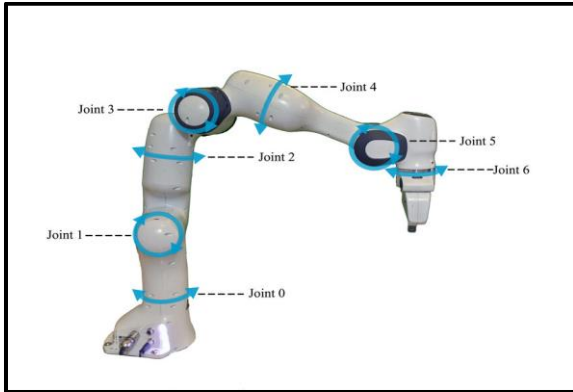


Figure 1

The paper is structured as follows. Section 3 and 4 describes the system architecture and implementation of the work packages, including robot kinematics, trajectory planning, and calibration procedures. Section 5 presents the methodology for calibration, data acquisition, model registration, and needle path planning. Section 6 reports the experimental results, focusing on calibration accuracy, registration performance, and insertion precision. Section 7 discusses the challenges encountered, design choices, and potential improvements. Further, Section 7 concludes the paper and outlines directions for future work in robotic needle placement systems.

3. Robot Kinematics

3.1 Frame Assignment and Forward Kinematics

The Franka Emika Panda is a 7-degree-of-freedom (DOF) manipulator. To systematically describe its kinematic structure, we adopted the Modified Denavit–Hartenberg (MDH) convention. The coordinate frames were assigned to each joint according to the standard procedure (see Figure 2), and the corresponding MDH parameters are summarized in Table 1 (see attached figure for frame assignment).

Using these parameters, the homogeneous transformation matrix from the base frame to the end-effector frame can be expressed as:

$${}^0T_7 = \prod_{i=1}^7 {}^{i-1}T_i$$

where ${}^{i-1}T_i$ represents the individual joint transformation derived from the MDH parameters. This formulation provides the forward kinematics (FK), mapping a set of joint angles

$$\mathbf{q} = [q_1, q_2, \dots, q_7]^T$$

to the Cartesian pose of the end-effector in SE(3), represented by a rotation matrix $R \in SO(3)$ and a position vector $p \in R^3$.

The FK implementation was validated by computing end-effector poses from given joint configurations and cross-verifying with the robot's state feedback available in ROS via the `/joint_states` and `/tf` topics.

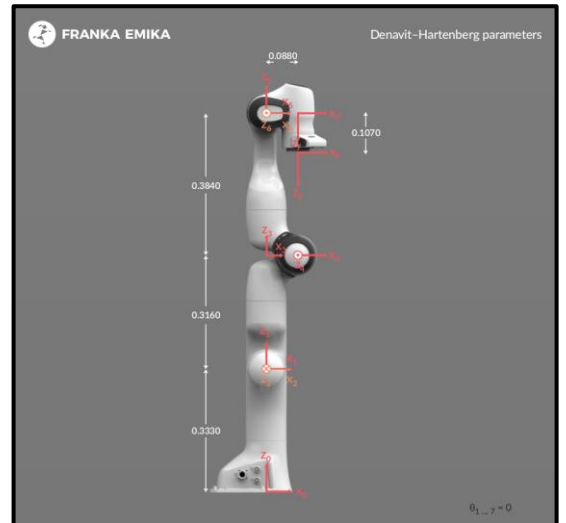


Figure 2

Joint	q_i (rad)	a_i (m)	d_i (m)	α_i (rad)
1	q_1	0.3330	0	0
2	q_2	0	0	$-\pi/2$
3	q_3	0.3160	0	$\pi/2$
4	q_4	0	0.0825	$\pi/2$
5	q_5	0.3840	-0.0825	$-\pi/2$
6	q_6	0	0	$\pi/2$
7	q_7	0	0.0880	$\pi/2$
Flange	0	0.1070	0	0

Table 1

3.2 Inverse Kinematics

The inverse kinematics (IK) problem is to determine the joint configuration $\mathbf{q}$ that realizes a desired end-effector pose (R_d, p_d). Unlike FK, the IK problem is more challenging due to nonlinearity, redundancy, and the possibility of multiple or no solutions. IK approaches can be broadly classified into two categories:

Analytical methods – These provide closed form solutions by solving

the kinematic equations algebraically. While exact and efficient for certain robots (e.g., 6-DOF manipulators with spherical wrists), analytical derivation becomes increasingly complex for redundant or non-standard architectures such as the 7-DOF Panda.

Numerical methods – These rely on iterative updates to approximate a feasible solution. They are more general and flexible, especially for redundant manipulators, but may converge slowly or to local minima. Common approaches include Jacobian-based iterative schemes and optimization-based methods.

Given the redundancy of the Panda robot and the flexibility required for needle insertion tasks, we implemented a Jacobian-based Incremental Inverse Kinematics method. This approach computes small joint angle updates using the pseudo-inverse of the Jacobian matrix J :

$$\Delta \mathbf{q} = \mathbf{J}^\dagger \Delta \mathbf{x}$$

where $\Delta \mathbf{x}$ is the task-space error between the current and desired end-effector pose, and $\mathbf{J}^\dagger$ is the Moore–Penrose pseudo-inverse of the Jacobian. Iteratively applying these updates drives the end-effector towards the target pose. This method yields a minimum-norm solution, which naturally resolves redundancy by selecting the joint configuration with the smallest overall movement. It is computationally efficient, robust to redundancy, and suitable for real-time trajectory generation.

The implementation was carried out in Python and tested in simulation. The computed joint configurations were verified by forward kinematics, ensuring that the desired end-effector poses were accurately reproduced.

4. Trajectory Planning

4.1 Path Planning and Trajectory Generation

The goal of trajectory planning in this project was to compute smooth, collision-free motions that guide the robot from an arbitrary initial configuration to a desired target pose, as determined from calibration and model registration. The process consisted of two main phases:

Path Planning – A feasible path was generated in the configuration space, ensuring that joint limits were respected and obstacles in the environment (obtained from the point cloud representation of the phantom) were avoided.

Trajectory Generation – The path was converted into a smooth, time-parameterized trajectory that defined joint positions, velocities, and accelerations for execution on the robot.

To achieve smooth and dynamically consistent motion, quintic polynomial interpolation was employed. This method satisfies six boundary conditions (position, velocity, and acceleration at both start and end), producing trajectories with continuous accelerations and avoiding abrupt changes in motion.

4.2 Two-Stage Planning Scheme

The trajectory planning task was divided into two stages to address the specific requirements of needle insertion:

Stage 1: Initial to Intermediate Pose (Configuration Space) The robot was first guided from its initial configuration to an intermediate pose located just outside the phantom. This intermediate point was selected such that the subsequent needle insertion could follow a straight-line trajectory. Planning in this phase was performed in the configuration space using quintic polynomial interpolation, ensuring smooth and feasible motion while respecting joint limits.

Stage 2: Intermediate to Target Pose (Task Space) From the intermediate point, the needle was advanced to the final target pose along a straight line in Cartesian space. This task-space trajectory was generated by interpolating the needle tip position while maintaining a fixed orientation, thereby ensuring accurate and clinically relevant insertion. The interpolated poses were converted to joint configurations through inverse kinematics.

This two-stage approach effectively combined the flexibility of configuration space planning with the precision of task space motion, enabling safe positioning and accurate insertion.

4.3 Optimization-Based Joint Trajectories

To improve the quality of motion, trajectory generation was further formulated as a constrained optimization problem. The objective was to minimize jerk (third derivative of position) while satisfying the following constraints:

Boundary conditions: prescribed joint positions, velocities, and accelerations at the start and end configurations.

Path constraints: adherence to the robot's physical joint limits.

This optimization framework ensured time-efficient, smooth trajectories that were dynamically feasible and safe for execution. The optimized planner was successfully tested in simulation and later implemented on the real robot, where it achieved smooth and precise motion without constraint violations.

4.4 Implementation and Validation

The planned trajectories were validated first in the simulation environment using RViz, confirming feasibility and collision-free motion. Subsequently, the trajectories were executed on the real Franka Emika Panda robot. During early tests, some discrepancies between feedback topics introduced minor execution issues, but these were resolved, resulting in stable and accurate performance.

The straight-line insertion motion in task space was also successfully integrated with the quintic polynomial joint-space trajectory generator, producing a unified trajectory planning framework. The combined system enabled the robot to move smoothly from an arbitrary initial pose to the selected intermediate point, and then to the final target location along the desired insertion path.

5. Calibration and Registration

5.1 Camera calibration

For the Azure Kinect sensor, calibration was performed on both the RGB and depth cameras to accurately characterize their geometric properties. The process involved estimating the intrinsic parameters (focal length, principal point, and lens distortion) for each camera independently, followed by determining their extrinsic parameters, which define the spatial relationship between the RGB and depth sensors. A stereo calibration step was then carried out to compute the rigid transformation between the two lenses, enabling precise alignment of depth information with the corresponding RGB images. This calibration ensured accurate sensor fusion and provided a reliable basis for generating metrically consistent 3D reconstructions and point clouds.

5.2 Hand-eye calibration

Hand-eye calibration is a fundamental step in robotic vision systems where a camera is rigidly mounted on the robot's end-effector (eye-in-hand configuration). The objective is to determine the rigid transformation between the robot's end-effector frame and the camera frame. This calibration allows visual information from the camera to be expressed consistently in the robot base coordinate system, enabling sensor-guided manipulation and accurate point cloud alignment.

5.2.1 Mathematical Formulation

The classical hand-eye calibration problem is represented by the matrix equation:

$$\mathbf{A}_i \mathbf{X} = \mathbf{X} \mathbf{B}_i$$

where:

- $\mathbf{A}_i \in \text{SE}(3)$: motion of the robot's end-effector between two poses (obtained from forward kinematics).
- $\mathbf{B}_i \in \text{SE}(3)$: corresponding motion of the camera between two poses (obtained from camera calibration pattern detections).
- $\mathbf{X} \in \text{SE}(3)$: unknown rigid transformation from the end-effector

frame to the camera frame, which is to be estimated.

This formulation assumes that both robot kinematics and camera pose estimation provide consistent motion trajectories.

5.2.2 Methodology in Our Work:

- **Robot Motion Capture:** The joint states of the 7-DOF Franka Panda robot were recorded from the ROS topic. Using the robot's Denavit-Hartenberg parameters, the forward kinematics were computed to obtain the transformation matrices from the base frame to the end-effector.]
- **Camera Motion Estimation:** RGB and depth images were extracted from the Azure Kinect camera. A calibration target (checkerboard pattern) was detected in these images, and the relative motion of the camera between successive frames was estimated using OpenCV pose estimation methods.
- **Solving the Hand-Eye Equation:** Using the pairs (A_i, B_i) , the hand-eye calibration problem was solved with the Tsai-Lenz and Daniilidis algorithms implemented in OpenCV. The result was the transformation matrix (A_i, B_i) , representing the fixed spatial relationship between the camera and the robot end-effector.
- **Application in the Project:**
 - The estimated transformation X was used to transform all camera poses into the robot base frame.
 - This enabled consistent stitching of 3D point clouds from multiple viewpoints.
 - The aligned point clouds were later registered against a reference CAD model (Skeleton.stl) for further analysis.

5.2.4 Assumptions and Considerations

- The camera was assumed to be rigidly mounted to the robot's end-effector without mechanical slippage.
- Calibration accuracy depends on having sufficiently diverse robot motions covering different orientations.
- Noise in joint encoder measurements and camera pose estimation contributes to residual calibration error.

5.3 Point Cloud registration

Point cloud processing was performed to generate a unified 3D model from the Azure Kinect sensor. The point cloud data was acquired from the ROS topic /point2cloud and imported into Open3D for processing. Using the camera calibration and the hand eye data, a metrically accurate point cloud was constructed. The ICP algorithm was applied where necessary to refine the alignment of successive frames, ensuring a coherent and smooth reconstruction. The final point cloud provided a precise 3D representation of the patient, which served as the basis for planning and guiding the needle injection procedure.

6. Experimental Setup and Validation

6.1 Simulation Environment

The development and initial testing of the proposed system were carried out in a simulation environment. The robot was visualized in RViz within the ROS framework, where individual nodes were implemented to control robot motion, perform trajectory generation, and verify kinematic computations. The smoothness of motion was evaluated by plotting joint positions, velocities, and accelerations during execution, ensuring that trajectories satisfied the continuity requirements established in the planning stage. This allowed for rapid testing and debugging of the algorithms prior to deployment on the real system.

6.2 Real-World Setup

Following validation in simulation, the algorithms were deployed on the Franka Emika Panda robot in the laboratory environment (see Figure 3). A chest phantom was placed in front of the robot to act as the test object, as shown in Figure X. The Microsoft Azure Kinect depth camera, mounted on the robot's end-effector, was used to capture 3D point cloud data of the phantom.

The same control nodes used in simulation were executed on the physical robot, and smooth, stable trajectories were observed. The

robot successfully followed planned motions, transitioning from arbitrary initial poses to target configurations without abrupt behavior.

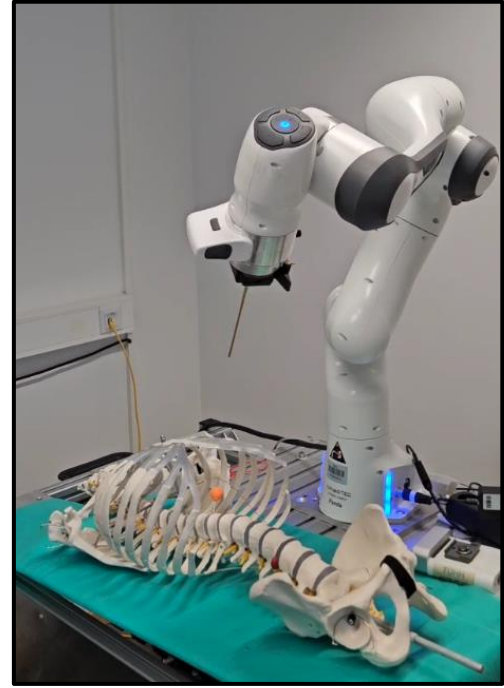


Figure 3

6.3 Model Registration and Target Selection

To validate the integration of perception and motion planning, point clouds of the phantom were captured from multiple viewpoints. These were transformed into the robot base frame using the hand-eye calibration and the transformation between the end-effector and the base frame. The registered point cloud enabled accurate representation of the phantom relative to the robot's workspace. Within the high-resolution model, a target point was selected (highlighted in red in Figure Y). Based on this target, a collision-free insertion orientation was determined. This provided the final desired pose (position and orientation) for the robot's end-effector.

6.4 Validation of Motion Execution

The robot was commanded to move to the computed target configuration, following the two-stage trajectory planning scheme described earlier. The initial motion was performed in configuration space up to the intermediate pose, after which the end-effector followed a straight-line trajectory in task space toward the target. The robot was able to successfully reach the goal configuration, demonstrating the feasibility of the proposed method for image-guided robotic needle placement.

Some discrepancies and limitations were observed during real-world execution, which are discussed in Section 6 (Discussion and Future Work)

$$\mathbf{P}_{\text{base}} = {}^{\text{base}}T_{\text{ee}} \cdot {}^{\text{ee}}T_{\text{cam}} \cdot \mathbf{P}_{\text{cam}}$$

Here:

${}^{\text{base}}T_{\text{ee}} \in SE(3)$ is the transformation from the robot base to the end-effector,

${}^{\text{ee}}T_{\text{cam}} \in SE(3)$ is the hand-eye calibration (transformation from end-effector to camera),

$\mathbf{P}_{\text{cam}} \in \mathbb{R}^3$ is a point in the camera coordinate frame,

$\mathbf{P}_{\text{base}} \in \mathbb{R}^3$ is the same point expressed in the robot base frame.

$${}^{\text{base}}T_{\text{target}} = \begin{bmatrix} R_{\text{target}} & \mathbf{p}_{\text{target}} \\ 0 & 1 \end{bmatrix},$$

where

$R_{\text{target}} \in SO(3)$ defines the collision-free orientation,

and

$\mathbf{p}_{\text{target}} \in \mathbb{R}^3$ is the position of the selected target point in the base frame.

7. Discussion and Future Work

The developed image-guided robotic needle placement system successfully integrated kinematic modeling, trajectory planning,

calibration, and real-world execution. The experiments demonstrated smooth robot motion both in simulation and on the real Panda robot, validating the feasibility of the proposed framework for controlled needle insertion tasks.

Although the system performed reliably, several aspects remain to be addressed for practical deployment in clinical settings.

First, in the current implementation, the chest phantom was not explicitly modeled as an obstacle during trajectory planning. Instead, trajectories were manually defined to avoid collision with the phantom. While this was sufficient for laboratory validation, real medical applications require precise obstacle modeling to ensure patient safety. Incorporating detailed anatomical models into the planning process would allow the robot to autonomously generate collision-free paths that adapt to individual patient anatomy.

Second, point cloud registration was limited to aligning scans obtained during robotic scanning. The registration of intraoperative data to preoperative patient models, such as high-resolution CT scans, was not fully realized. This step is crucial for clinical relevance, as it eliminates the need to repeatedly acquire point clouds and enables direct integration with medical imaging modalities. Reliable registration between intraoperative and preoperative data would significantly improve both the efficiency and accuracy of robotic interventions.

Future work will therefore focus on:

- Developing robust obstacle-aware trajectory planning methods that incorporate detailed patient-specific anatomical models.
- Extending the registration pipeline to enable seamless alignment of Kinect-based scans with preoperative CT data.
- Evaluating the system on more complex phantom setups or deformable tissue models to better approximate clinical conditions.

By addressing these limitations, the proposed framework could be transformed into a clinically viable robotic needle placement system, capable of improving the precision, safety, and efficiency of minimally invasive procedures

8. Work Contribution

Name	Work Contribution
Ashutosh Goyal	• Implementation and validation of Forward and Inverse Kinematics • Trajectory planning in configuration space as well as in task space. • Scanning bag creation. • Removing bugs from shifting to simulation to real world (sim 2 real gap) • Helping perception team in calibration and registration task. • Presentation and Report formulation.
Ravi Prakash Dhorajia	• Implementation and validation of Forward and Inverse Kinematics • Trajectory planning in configuration space as well as in task space. • Scanning bag creation. • Removing bugs from shifting to simulation to real world (sim 2 real gap) • Helping perception team in calibration and registration task. • Presentation and Report formulation.
Shashank Somasundaram	• Implementation and validation of Inverse Kinematics. • Extracted images from Scanning bag, used for Calibration. • Helped in removing bugs during Trajectory Planning. • Helped Perception team in calibration and cloud point registration.

Sharjeel Ahmad	• Performed camera calibration for both RGB and depth lenses to correct distortion and improve accuracy. • Estimated and extracted intrinsic (focal length, principal point, distortion) and extrinsic (pose, orientation) parameters. • Generated an Open3D point cloud from point2cloud topic for 3D reconstruction. • Applied FPFH features to compute local descriptors and obtain an initial rough alignment between models. • Employed RANSAC-based global registration for robust alignment under noise and outliers. • Refined the registration using the Iterative Closest Point (ICP) algorithm for precise alignment. • Visualized and validated alignment results using Open3D's 3D rendering tools.
Vinit Trivedi	• Performed Hand-Eye Calibration for the Franka Panda robot with the Azure Kinect camera to establish the transformation between end-effector and camera. • Extracted joint states and camera poses from ROS bag files and utilized them for calibration. • Implemented calibration pipeline using OpenCV methods and validated the obtained transformation. • Helped in generating 3D point clouds from RGB-D data. • Contributed to stitching point clouds using calibrated transformations. • Assisted in visualizing point clouds in RViz for inspection and validation.

9. References

- Trabelsi, Amir & Sandoval Arévalo, Juan Sebastián & Moncef, Ghiss & Laribi, Med Amine. (2021). Development of a Franka Emika Cobot Simulator Platform (CSP) Dedicated to Medical Applications. 10.1007/978-3-030-75259-0_11.
- Reed, Alec & Albin, Doncey & Pasricha, Anuh & Roncone, Alessandro & Heckman, Christoffer. (2024). Transformer-based Learning Models of Dynamical Systems for Robotic State Prediction. 10.21203/rs.3.rs-3919154/v1.
- <https://franka.de/>
- <https://franka.de/documents>
- Gaz, Claudio & Cagnetti, Marco & Oliva, Alexander & Giordano, Paolo & Luca, Alessandro. (2019). Dynamic Identification of the Franka Emika Panda Robot With Retrieval of Feasible Parameters Using Penalty-Based Optimization. IEEE Robotics and Automation Letters. PP. 1-1. 10.1109/LRA.2019.2931248.